

GPTVD: Vulnerability Detection and Analysis Method Based on LLM's Chain of Thoughts.

Yinan Chen

School of Software Engineering, Sun Yat-sen University

Yuan Huang

School of Software Engineering, Sun Yat-sen University

Xiangping Chen

School of Journalism and Communication, Sun Yat-sen University

Pengfei Shen

School of Software Engineering, Sun Yat-sen University

Lei Yun

yl@ceprei.com

School of Software Engineering, Sun Yat-sen University

Research Article

Keywords: Vulnerability detection, Large language model, ICL, COT, Prompt engineering

Posted Date: April 21st, 2025

DOI: <https://doi.org/10.21203/rs.3.rs-6275325/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

GPTVD: Vulnerability Detection and Analysis Method Based on LLM’s Chain of Thoughts.

Yinan Chen¹, Yuan Huang¹, Xiangping Chen³, Pengfei Shen^{1,2},
Lei Yun^{1,2*}

¹School of Software Engineering, Sun Yat-sen University, Zhuhai, Guangdong, China.

²Key Laboratory Ministry of Industry and Information Technology, China Electronic Product Reliability and Environment Testing Research Institute, Guangzhou, Guangdong, China.

³School of Journalism and Communication, Sun Yat-sen University, Guangzhou, Guangdong, China.

*Corresponding author(s). E-mail(s): yl@ceprei.com;

Contributing authors: chenyn273@mail2.sysu.edu.cn;

huangyuan5@mail.sysu.edu.cn; chenxp8@mail.sysu.edu.cn;

shen_pf@qq.com;

Abstract

Purpose: Traditional vulnerability detection methods based on machine learning (ML) and deep learning (DL) primarily focus on coarse-grained predictions, often lacking precise localization and interpretability regarding the root causes of vulnerabilities. The growing availability of open-source vulnerability databases calls for advanced methods that can reason about vulnerabilities at a finer slice-level granularity. GPTVD, which leverages large language models’ (LLMs) in-context learning (ICL) and chain-of-thought (COT) reasoning capabilities. The goal is to enhance both detection performance and explainability. **Methods:** GPTVD extracts threat code slices through static code analysis, focusing on data and control dependencies. Positive and negative samples are clustered based on heuristic features, and representative samples are manually annotated with reasoning processes to build COT prompts. These prompts are combined with target samples to form LLM input queries, enabling slice-level vulnerability inference and explanation using ChatGPT. The method was evaluated on 3,512 threat code slices from a public dataset. **Results:** GPTVD achieved superior performance compared to state-of-the-art methods, with 90.72% accuracy,

86.71% precision, and 96.39% recall. Ablation studies confirm that clustering-based prompt selection, explicit threat code slices, and human expert reasoning significantly improve detection effectiveness and interpretability. **Conclusion:** GPTVD demonstrates that combining static code analysis with LLM-based COT reasoning can effectively detect vulnerabilities at the slice level with high accuracy and interpretability.

Keywords: Vulnerability detection, Large language model, ICL, COT, Prompt engineering

1 Introduction

The software development process generates many software vulnerabilities, as even experienced developers are prone to making errors [1]. These vulnerabilities pose a threat to software quality and can result in the malfunctioning of software, crashes, or data damage. The widespread use of open-source code and code cloning further propagates these flaws, leading to severe security issues in the software industry and the field of cybersecurity [2]. Therefore, establishing effective vulnerability detection mechanisms is of utmost importance. Such mechanisms can help improve software quality, ensure software security, and reduce software maintenance costs.

Existing code vulnerability detection methods can be broadly categorized into coarse-grained detection methods and fine-grained detection methods [1].

Coarse-grained methods detect the presence of vulnerabilities in source code, functions, or code slices but cannot pinpoint the specific vulnerable statements. Some methods represent the source code as token sequences and utilize RNN or Transformer [3] models for sequence classification tasks, such as VulDeePecker [4]. Some methods transform the source code into graphs and employ GNN models for graph classification, with Devign [5] being a representative example. Although coarse-grained methods achieve good detection results, they fail to locate the exact vulnerability locations, which imposes an additional workload on code developers.

Fine-grained methods aim to predict vulnerabilities at the statement level, thus helping code developers quickly locate vulnerabilities. Some of these methods leverage the interpretability of deep learning attention mechanisms to identify the statements that contribute the most to the predictions, such as IVDetect [6]. Another category of methods directly trains models using statement-level labels for prediction, as seen in LineVD [7]. However, current fine-grained methods suffer from shortcomings in code representation, embedding learning, and model design, leading to suboptimal code line localization performance. Additionally, even if the vulnerable statements can be successfully predicted, the lack of strong interpretability necessitates further analysis by developers to determine the causes of vulnerabilities, resulting in additional workload.

To address these issues, this paper proposes GPTVD, an LLM-based slice-level vulnerability reasoning detection method. GPTVD adopts the paradigm of in-context

learning (ICL) [8], which allows the Large Language Model (LLM) to directly generate expected answers by providing relevant demonstration examples in the prompt. Thus, GPTVD can learn from demonstration examples how to reason about the presence of vulnerabilities by analyzing code data dependencies and control dependencies. Specifically, GPTVD utilizes ChatGPT, a popular pre-trained language model, as the base model. It extracts data dependency and control dependency slices of threat code through static code analysis. By clustering, GPTVD selects positive and negative prompt samples and constructs reasoning processes regarding the presence of vulnerabilities for prompt samples manually. This forms the prompt used to detect whether the target sample contains vulnerabilities, leveraging the LLM’s contextual learning and reasoning abilities. GPTVD achieves slice-level vulnerability detection and provides a detailed analysis of vulnerability causes. GPTVD has been evaluated on a publicly available dataset consisting of 3,512 test threat code slices. The results demonstrate that GPTVD achieves (1) an accuracy of 85.71%, (2) a recall rate of 96.39%, and (3) an F1 score of 0.913 in vulnerability detection. Compared to existing methods, GPTVD exhibits state-of-the-art performance. Additionally, the vulnerability reasoning processes generated by GPTVD provide a detailed analysis of the causes of vulnerabilities. The main contributions of this paper are as follows:

1. We combine vulnerability detection with LLM and achieve better performance in vulnerability detection. By leveraging the contextual learning and reasoning abilities of LLM, we provide natural language analysis of vulnerability causes for the first time, which enhances interpretability and helps developers locate and fix vulnerabilities.
2. We construct a comprehensive Chain of Thoughts (COT) for vulnerability detection based on data dependencies and control dependencies.
3. We significantly reduce the manual construction of example samples COT by selecting representative vulnerability detection prompt samples through clustering.
4. We have open-sourced the test results, COT, and prompt formats, facilitating further research by relevant researchers.

The remaining sections of the paper are organized as follows. Section 2 introduces existing vulnerability detection methods and the application of LLM in SE. Section 3 provides an overview of what threat code slices and COT are. Section 4 provides a detailed description of the design of GPTVD. Section 5 discusses the research questions and presents experimental results and analysis. The failures of GPTVD are introduced in Section 6. Section 7 introduces factors that may affect the validity of GPTVD. Finally, we make a conclusion in Section 8 to summarize the paper and outline future research directions.

2 Related Work

2.1 Vulnerability detection

GPTVD combines static code analysis with LLM for vulnerability detection. Existing static vulnerability detection methods can be classified into two types: rule-based and machine learning-based. Rule-based tools like Flawfinder and Checkmarx rely on

predefined rules designed by experts to detect vulnerabilities. However, these methods often suffer from high false positives, requiring engineers to perform tedious work in verifying false alarms [1].

Machine learning-based methods enable models to automatically learn the capability of detecting vulnerabilities from a large amount of data. Based on the detection granularity, machine learning-based methods can be further categorized into coarse-grained and fine-grained approaches [1].

Coarse-grained methods detect vulnerabilities in source codes, functions, or code slices without pinpointing the vulnerable statements. Examples of coarse-grained methods include VulDeePecker [4] and μ VulDeePecker [9], which treat source code as token sequences and employ sequence models such as LSTM [10] or Transformer [3] for sequence classification tasks. VulDeePecker constructs code gadgets by analyzing data dependency of library/API function calls and utilizes LSTM for vulnerability detection. μ VulDeePecker extends VulDeePecker to detect multiple classes of vulnerabilities. SySeVR [11] enhances VulDeePecker by introducing more syntactic features and control dependencies in code representations and is more effective than rule-based detectors such as Flawfinder and Checkmarx. Another coarse-grained approach such as DeepWukong [12], FUNDED [13], and Devign [5] transforms source code into graphs and applies graph neural networks as detectors. Specifically, Devign [5] represents source code as a Code Property Graph (CPG) with natural code sequence edges and uses GGNN [14] as the detection model. DeepWukong [12] performs similarly to Devign and proposes XFG (a subgraph of PDG) as the code representation, employing k-GNN to learn vulnerability features from the graph.

Fine-grained methods aim to provide statement-level vulnerability predictions. Some works propose the use of interpretable deep-learning mechanisms for fine-grained predictions. Specifically, VulDeeLocator [15] designs a vulnerability detection network that utilizes attention mechanisms to select vulnerable code lines. DeepLineDP [16] also uses attention mechanisms to indicate the likelihood of code tokens causing vulnerabilities. In contrast to attention-based models, Zou et al. [17] propose a heuristic search-based framework to identify tokens that contribute significantly to predictions. IVDetect [6] leverages an advanced interpretable GNN model called GNNEExplainer [18] to learn edge masks and generate the most decisive subgraphs in Program Dependence Graphs (PDG). Leveraging fine-grained labels, LineVD [7] directly trains and predicts vulnerabilities at the line level in a fully supervised manner.

GPTVD differs from existing methods as it not only detects the presence of vulnerabilities at the coarse-grained level but also provides fine-grained localization of the vulnerable code lines and analyzes the causes leading to vulnerabilities. Compared to previous works, we apply the COT of large models and contextual learning capabilities to vulnerability reasoning, achieving vulnerability detection at the slice level.

2.2 LLM for software engineering

The application of Large Language Models (LLMs) in software engineering is primarily achieved through pretraining and fine-tuning [19, 20]. In the pretraining phase, a large language model is trained on a large-scale code dataset to acquire a semantic understanding of source code. Therefore, LLM can be applied to many code tasks. For

example, more accurate and intelligent code searches and recommendations can be achieved. LLMs can learn semantic representations of code snippets and utilize these representations to generate more concise and readable code summaries.

Some researchers have also explored the application of LLMs in more specific downstream tasks using their contextual learning and reasoning capabilities. For example, Peng et al. proposed a new automatic type inference method called TYPEGEN [21]. In general, TYPEGEN adopts a generative learning paradigm that incorporates domain knowledge from static analysis into the language model to improve the effectiveness of type inference. Specifically, TYPEGEN consists of four stages: 1) Code Slicing, extracting code snippets based on type dependency graphs; 2) Type Hint Collection, gathering user-defined and third-party library types; 3) Chain-of-Thought Hint Construction, translating static analysis inference steps into hints; and 4) Type Generation, using contextual learning methods combined with several example hints to predict the type of the target variable. Xu et al. [22] proposed UniLog, an automatic logging framework based on the in-context learning (ICL) paradigm of Large Language Models (LLMs). UniLog can generate complete log statements for a given code snippet, including determining log locations, predicting log verbosity, and generating log messages. It designs line-aware hints and utilizes the kNN algorithm to select demonstration examples for hints. Experimental results on real-world datasets demonstrate that UniLog outperforms existing methods and achieves state-of-the-art performance in all logging subtasks. Inspired by them, we combine static code analysis with the inference capabilities of large models, enabling LLMs to perform slice-level vulnerability detection by learning vulnerability inference methods based on data flow and control flow analysis through ICL.

3 Technical background

3.1 Threat code slice

Slice-level vulnerability detection is built upon threat code slices. Here we give the definition of threat code slices: a threat code slice consists of a set of program statements (i.e., code lines) that are related through data dependencies or control dependencies. To generate threat code slices, we adopt the approach of VulDeePecker [4], which extracts code slices from the source code starting from key points (i.e., threat code). Intuitively, key points can be seen as the center of vulnerabilities or code that implies the vulnerabilities. We will provide a detailed explanation of the key points considered in our work in the Section 4. Starting from the key points, we extract statements from the source code that have data dependencies and control dependencies with the key points. These statements are then arranged in the order of program execution, forming threat code slices related to the key points. As shown in Fig. 1, in this example source code snippet, the key point (i.e., threat code) is line 80. The statements that have control dependencies with it are at lines 68, 77, and 85, while the statements that have data dependencies with it are at lines 70, 71, 72, 84, and 87. We arrange them according to their actual execution order in the program to form a threat code slice.

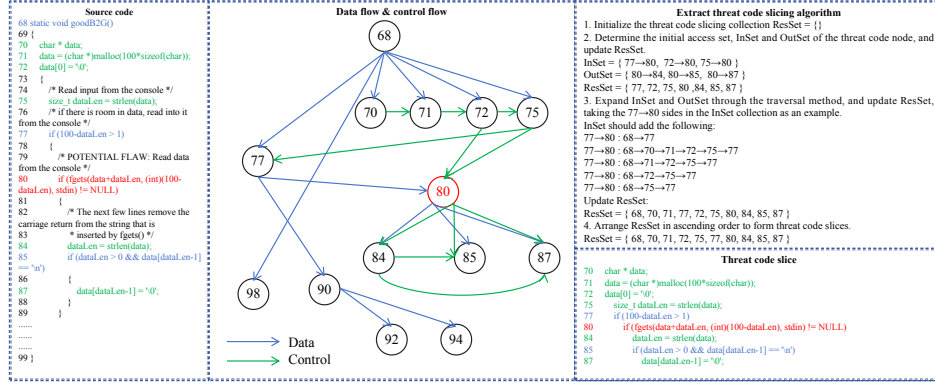


Fig. 1 Threat code slice extraction.

3.2 COT

With the significant increase in parameter size and training corpus size, LLM (Large Language Model) has become increasingly powerful. Researchers have found that the capabilities of LLM can be significantly enhanced with the use of text prompts. Studies have shown that the essence of Instruction-based in-context learning (ICL) is the ability of LLM to perform complex reasoning based on a large amount of training data, enabling LLM to simulate the expected answers to query (i.e., target samples) based on the demonstrated examples in the prompt. Specifically, in the paradigm of in-context learning, the prompt input to LLM typically consists of (1) instructions for a specific task, (2) several demonstrated examples containing ground truth and reasoning for the specific task, and (3) queries to LLM about target samples for the specific task, expecting appropriate answers.

As a new paradigm for LLM, in-context learning (ICL) has many inherent advantages. Firstly, since ICL does not require network structure exploration or the design of optimization objectives, LLM can adapt to new tasks with lower computational costs through task-specific prompts. Secondly, because all instructions and descriptions are in natural language, the interaction process between users and LLM is interpretable and easy to understand. Thirdly, due to the direct utilization of knowledge acquired from pre-training and information from prompt demonstrated examples, there is no significant gap between pre-training and downstream tasks. This approach avoids the problem of overfitting due to excessive reliance on specific downstream task data, allowing users to fully explore the knowledge of LLM. These advantages align perfectly with the requirements of vulnerability detection based on data flow and control flow analysis.

Considering the significant advantages of ICL, we propose GPTVD, which is the first attempt to perform vulnerability inference detection tasks within the ICL paradigm. Specifically, GPTVD no longer requires explicit learning of vulnerability detection from a training process that consumes a large amount of vulnerability data. Instead, it utilizes the analogical reasoning ability of LLM to directly provide the data

flow and control flow analysis process of threat code slices from the demonstrated examples provided in the prompt and determine whether vulnerabilities exist.

4 Method

The overall architecture of GPTVD is shown in Fig. 2. Firstly, we locate C/C++ API library functions in the source code, considering them as key points (i.e., threat code). Then, we form the data dependencies and control dependencies graph of the source code through static code analysis and extract the statements that have data dependencies and control dependencies with the threat code, forming threat code slices. We consider slices with vulnerabilities as positive samples and those without vulnerabilities as negative samples. Next, we cluster the positive and negative sample slices separately based on heuristic features to obtain cluster center samples for both positive and negative samples. Human experts analyze the data flow and control flow of the cluster center samples to detect whether vulnerabilities exist. Finally, we treat the cluster center samples from the positive and negative clusters as demonstrated example samples in the prompt. By combining the demonstrated examples with queries (i.e., target samples), we form the LLM ICL paradigm prompt and input it to LLM to analyze whether vulnerabilities exist in the target samples.

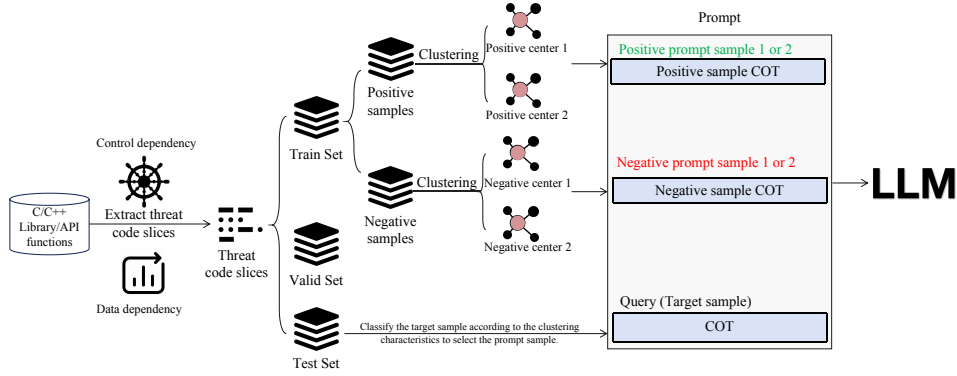


Fig. 2 The overall architecture of GPTVD.

4.1 Extract threat code slices

To extract threat code slices, we first need to identify key points, which are the threat code. This study focuses on buffer overflow vulnerabilities, and key points related to buffer overflow vulnerabilities include library/API function calls, arrays, and pointers. In this work, we specifically emphasize the effectiveness of using specific key points of library/API function calls to demonstrate the efficacy in vulnerability detection based on large-scale models. We consider 138 types of C/C++ library/API function calls,

Table 1 Threat C/C++ library/API function calls.

CWE-119	cin, getenv, getenvs, wgetenv, wgetenv s, catgets, gets, getchar, getc, getch, getche, kbhit, stdin, getdltext, getpass, scanf, fscanf, vscaf, vfscanf, istream.get, istream.getline, istream.peek, istream.read*, istream.putback, streambuf.sbumpc, streambuf.sgetc, streambuf.sgetn, streambuf.snextc, streambuf.sputbackc, SendMessage, SendMessageCallback, SendNotifyMessage, PostMessage, PostThreadMessage, recv, recvfrom, Receive, ReceiveFrom, ReceiveFromEx, Socket.Receive*, memcpy, wmemcpy, memccpy, memmove, wmemmove, memset, wmemset, memcmp, wmemcmp, memchr, wmemchr, strncpy, strncpy*, lstrcpyn, tcsncpy*, mbsnbcpy*, wcsncpy*, wcsncpy, strncat, strncat*, mbsncat*, wcsncat*, bcopy, strcpy, lstrcpy, wcsncpy, tcscpy, mbscpy, CopyMemory, strcat, strlen, lstrlen, strchr, strcmp, strcoll, strcspn, strerror, strlen, strpbrk, strrchr, strspn, strstr, strtok, strxfrm, readlink, fgets, sscanf, swscanf, sscanffs, swscanf, printf, vprintf, swprintf, vsprintf, asprintf, vasprintf, fprintf, sprintf, snprintf, snprintf*, snwprintf*, vsnprintf, CString.Format, CString.FormatV, CString.FormatMessage, CStringT.Format, CStringT.FormatV, CStringT.FormatMessage, CStringT.FormatMessageV, syslog, malloc, Winmain, GetRawInput*, GetComboBoxInfo, GetWindowText, GetKeyNameText, Dde*, GetFileMUI*, GetLocaleInfo*, GetString*, GetCursor*, GetScroll*, GetDlgItem*, GetMenuItem*
CWE-399	free, delete, new, malloc, realloc, calloc, alloca, strdup, asprintf, vsprintf, vasprintf, sprintf, snprintf, snwprintf, vsnprintf

as shown in TABLE 1. Firstly, we locate these key points in the source code. Then, we use static code analysis tools to generate data flow and control flow graphs of the source code, as shown in Fig. 1. Finally, we extract the code lines related to data flow and control flow concerning the key points using graph traversal algorithms and arrange them into threat code slices according to the execution order of the program.

4.2 Prompt construction

The construction of prompts is a crucial part of in-context learning, and designing appropriate prompts requires consideration of various aspects, such as how to design question instructions, how to design prompt templates, and how to select demonstration examples. In GPTVD, we designed a prompt template at the granularity of code lines, which includes human reasoning processes. Considering that the changes in data flow and control flow for vulnerability detection are complex in practice and cannot be automated to form comprehensive analysis statements for data dependencies and control dependencies, GPTVD uses COTs that are analyzed sentence by sentence by human experts for the data flow and control flow of threat code slices. However,

manually constructing reasoning COTs on the entire set of candidate prompt samples is labor-intensive and not practical. Moreover, to ensure the generalization of demonstration example samples, we extracted heuristic features from the candidate example slices in the train set and selected representative demonstration example samples through clustering. We constructed COTs for them by human experts. These strategies enable LLM to perform vulnerability inference detection tasks in a more targeted manner. Next, we will provide a detailed explanation of the prompt construction method.

4.2.1 COT format

We aim to utilize LLM’s contextual learning and analogical reasoning capabilities, using LLM analyze the data flow and control flow of threat code slices like the prompt demonstration example to determine if vulnerabilities exist. To achieve this goal, we designed COT separately for the positive and negative prompt samples. COT represent the step-by-step reasoning process of determining if a sample contains vulnerabilities.

As shown in Fig. 3, we first provide the source code snippet with line numbers. Then, the threat code slice generated before is presented as an explicit prompt. Next, we indicate the line where the threat code resides in the sample. Finally, the human expert analysis process regarding the data flow and control flow of the threat code is described.

To achieve a detailed analysis process, human experts utilize the threat code slices generated during the code-slicing phase. As shown in Fig. 3, starting from the first statement of the threat code slice, the experts sequentially describe the definition and data changes of relevant variables, as well as the different execution paths that may be caused by the related control flow. The constructed COTs by GPTVD include a variable data flow and control flow. The data flow describes the data dependencies of variables, while the control flow describes the control dependencies that affect the execution paths of variables. When reaching key points (i.e., threat code), human experts make judgments based on the preceding descriptions to determine if vulnerabilities are present. Once all the statements of the threat code slice have been analyzed, the human experts provide a conclusion regarding the presence of vulnerabilities.

In the data flow and control flow analysis process constructed by human experts, each sentence begins with an ordinal number to indicate an inference step. All the sentences are connected in a backward traversal order to form a complete descriptive reasoning process. Fig. 3 illustrates two examples of the reasoning analysis process for positive and negative samples.

4.2.2 Clustering

Due to the labor-intensive and impractical nature of manually constructing the vulnerability analysis and reasoning process for demonstration example samples in GPTVD’s prompts on the entire set of candidate training samples, we extracted heuristic features from the candidate demonstration example slices and selected representative positive and negative demonstration example samples through clustering. Specifically, after obtaining the threat code slices, we divided them into train, valid, and test sets. The train set was used to generate demonstration example samples, the test set was

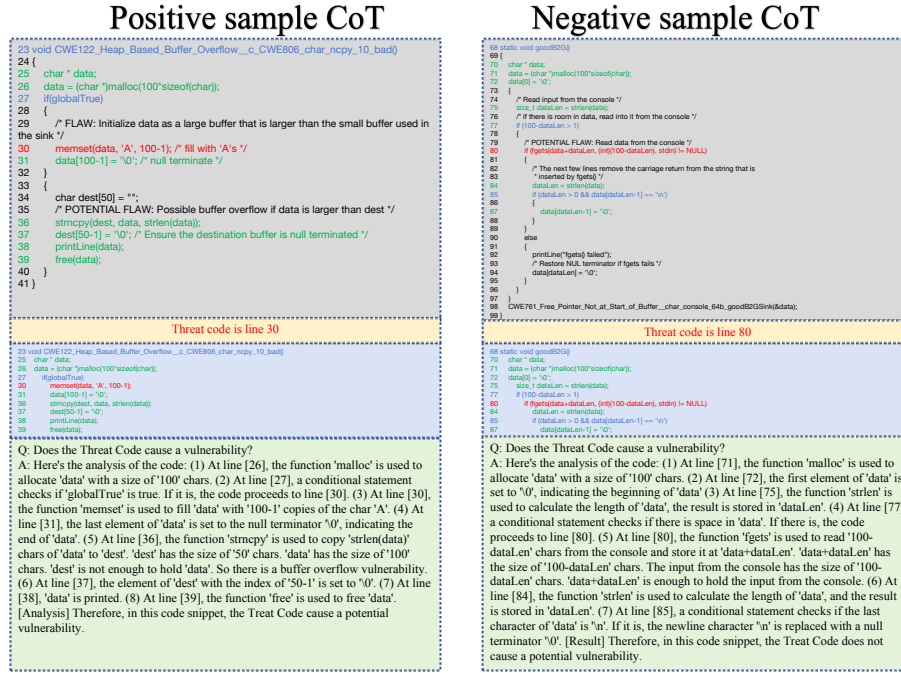


Fig. 3 The format of CoT.

Table 2 Heuristic features for clustering.

Slicing features	Description
Length of slice	Number of threat code slices
Code keywords	Number of IF, ELSE, WHILE and FOR keywords in the threat code slices
Semantic vector	Use pre-trained CodeBERT to convert slices into a 200-dimensional semantic vector.

used to generate target samples, and the valid set was used to validate the training of baseline deep learning methods.

For the samples in the train set, we further divided them into positive and negative samples and performed clustering based on heuristic features. We considered the length of the slice, code keyword features, and code semantic vector features, as shown in TABLE 2. The length of the slice is directly related to the length of data dependencies and control dependencies. The code keywords are associated with the complexity of control dependencies. Additionally, similar code slices may have similar vulnerability analysis and reasoning processes, so we also considered 200-dimensional semantic features. We applied the KNN algorithm to cluster the positive and negative train samples separately based on these features, with a cluster size set to 2. This resulted in two generalized positive prompt samples and two generalized negative prompt samples, respectively.

4.3 Prompt format

Considering the length restriction of LLM input, we provide one positive prompt sample and one negative prompt sample for each target sample when constructing the prompt. Specifically, given a target sample, we select the corresponding positive demonstration example and negative demonstration example based on the heuristic features of the target sample’s vulnerable code slice using the trained KNN model before. We combine the target sample’s query with these selected demonstration examples to form the final prompt, as shown in Fig. 4.

Positive prompt sample 1 or 2	
Positive sample COT	
Source code	
Threat code line	
Threat code slice	
Q: Does the Threat Code cause a vulnerability? A: [The analysis and reasoning process provided by human experts.], [result]	
Negative prompt sample 1 or 2	
Negative sample COT	
Source code	
Threat code line	
Threat code slice	
Q: Does the Threat Code cause a vulnerability? A: [The analysis and reasoning process provided by human experts.], [result]	
Query (Target sample)	
Target sample COT	
Source code	
Threat code line	
Threat code slice	
Q: Does the Threat Code cause a vulnerability? A: [To generate by LLM]	

Fig. 4 The format of prompt.

5 Evaluation

5.1 Dataset

We selected a publicly available dataset to validate the effectiveness and design rationality of GPTVD. This dataset is derived from SySeVR with marked ground truth labels. Considering the inference cost (time and resources) of ChatGPT, we focused on one type threat codes, C/C++ library/API function calls. Additionally, in practice, we removed a total of 1860 samples from the test set that exceeded the input length limit of ChatGPT. To ensure fairness, other baselines, and ablation experiments were also retrained and tested on this dataset. The dataset consists of a total of 64,403 samples, with sizes of 55,618 for the train set, 5,273 for the valid set, and 3,512 for the test set.

5.2 Experimental Setup

We chose ChatGPT as the base LLM and utilized the public API provided by OpenAI under the "gpt-3.5-turbo-0301" engine. During the inference process, LLM exhibits inherent randomness. To ensure consistent outputs for the same input, we set the temperature parameter to 0. This setting eliminates the randomization factor and ensures deterministic responses from LLM.

5.3 Evaluation Metrics

We use several metrics, including accuracy, precision, recall, and F1 score, to evaluate the effectiveness of vulnerability detection. Here are the concepts used in calculating these evaluation metrics. In this study, positive instances refer to samples with vulnerabilities, while negative instances refer to samples without vulnerabilities. TP (True Positive) represents the correctly predicted positive instances, where the true value of the data is positive, and the predicted value is also positive. TN (True Negative) represents the correctly predicted negative instances, where the true value of the data is negative, and the predicted value is also negative. FP (False Positive) represents the falsely predicted positive instances, where the true value of the data is negative, but it is incorrectly predicted as positive. FN (False Negative) represents the falsely predicted negative instances, where the true value of the data is positive, but it is incorrectly predicted as negative. With these concepts in mind, here are the meanings and calculation methods of each evaluation metric:

- **Accuracy:** it represents the proportion of correctly classified samples out of the total number of test samples..

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (1)$$

- **Precision:** it represents the proportion of samples predicted as positive that are actually positive.

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

- **Recall:** it represents the proportion of actual positive samples that are correctly predicted as positive out of all positive samples in the test set.

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

- **F1 score:** it is a weighted average of precision and recall, providing a balanced measure between the two metrics.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (4)$$

Table 3 The result of GPTVD compare to other slice-level vulnerability detection methods.

Method	Accuracy (%)	Precision (%)	Recall (%)	F1
VulDeePecker	87.27	84.21	91.94	0.879
GPTVD	90.72	86.71	96.39	0.913

5.4 Research Questions

To validate the effectiveness and design rationality of GPTVD for vulnerability detection, we have formulated several research questions. They aim to compare the effectiveness of GPTVD with other slice-level vulnerability detection methods and design ablation experiments to assess the design rationality of GPTVD’s prompts. The research questions are as follows:

- **RQ1:** How does GPTVD’s vulnerability detection performance compare to other slice-level vulnerability detection methods?
- **RQ2:** Is the selection of positive and negative prompt samples through clustering effective in GPTVD’s prompt design?
- **RQ3:** Is the explicit inclusion of threat code slices in GPTVD’s COTs effective?
- **RQ4:** Does the human expert analysis of data flow and control flow processes in GPTVD contribute to vulnerability detection?

5.5 Result Analysis

5.5.1 RQ1: How does GPTVD’s vulnerability detection performance compare to other slice-level vulnerability detection methods?

This research question aims to evaluate and compare GPTVD’s vulnerability detection performance with other slice-level vulnerability detection methods. In comparison to existing methods, we can assess the effectiveness and relative performance of GPTVD in detecting vulnerabilities.

We consider VulDeePecker, it was the first slice-level vulnerability detection technique but only considered data flow. VulDeePecker is based on deep learning. This research question evaluates based on accuracy, precision, recall, and F1 score to understand GPTVD’s detection capabilities. The results are shown in TABLE 3.

GPTVD is superior to VulDeePecker based on deep learning in all evaluation indicators, in which the recall rate is 5 percentage points higher. It shows that GPTVD can achieve a higher missed reporting rate than VulDeePecker. We give an example, as shown in Fig. 5. In this example, the size of “data” is 100, and the size of “dest” is 50, and when “memppy” is called to copy “data” to “dest”, because the size of “dest” is larger than the size of “data”, it will lead to vulnerabilities. GPTVD successfully detected the vulnerability by gradually analyzing the data flow.

```
# CWE121_Stack_Based_Buffer_Overflow__CWE806_char_alloca_memcpy_18.c
## Threat Code:
[memcpy] at line [36]
## Lines of (Data Flow and Control Flow) about Threat Code
23 void CWE121_Stack_Based_Buffer_Overflow__CWE806_char_alloca_memcpy_18_bad()
25 char * data;
26 char * dataBuffer = (char *)ALLOCA(100*sizeof(char));
27 data = dataBuffer;
31 memset(data, 'A', 100-1);
32 data[100-1] = '\0';
34 char dest[50] = "";
36 memcpy(dest, data, strlen(data)*sizeof(char));
37 dest[50-1] = '\0';

## Question: Is there a vulnerability in (Data Flow and Control Flow)?
## Answer: Here's the analysis of the code: (1) At line [26], 'dataBuffer' is allocated on the stack using the 'ALLOCA' macro, which allocates a buffer of size '100' chars. (2) At line [27], 'data' is assigned the address of 'dataBuffer', pointing to the allocated buffer on the stack. (3) At line [31], the function 'memset' is used to fill 'data' with '100-1' copies of the char 'A'. (4) At line [32], the last element of 'data' is set to the null terminator '\0', indicating the end of 'data'. (5) At line [34], 'dest' with a size of '50' chars is created. (6) At line [36], the function 'memcpy' is used to copy 'data' to 'dest'. 'dest' has the size of '50' chars. 'data' has the size of '100' chars. 'data' is larger than 'dest', causing a buffer overflow vulnerability. (7) At line [37], the last element of 'dest' is set to the null terminator '\0', ensuring the destination buffer is null terminated. (8) At line [38], 'data' is printed. [Analysis] In this code snippet, there is a potential vulnerability at line [36]. Therefore, the threat code [memcpy] at line [36] does cause a vulnerability.
```

Fig. 5 An example of GPTVD success but VulDeePecker failure.

Table 4 Experimental results of clustering effectiveness.

Method	Accuracy (%)	Precision (%)	Recall (%)	F1
Random selected samples	88.15	84.71	93.40	0.888
GPTVD	90.72	86.71	96.39	0.913

5.5.2 RQ2: Is the selection of positive and negative prompt samples through clustering effective in GPTVD’s prompt design?

We cluster similar code slices together based on heuristic features to capture similarities within each cluster and generate representative prompt samples, reducing the workload of human expert analysis.

To validate the effectiveness of this approach, we design comparative experiments where we select prompt samples randomly from the train set. The evaluation results are shown in TABLE 4.

It can be seen from the results, all evaluation indicators of GPTVD are higher than the method of random selection of prompt samples, which proves that cluster selection of prompt samples is generalized, which can improve the applicability of different types of samples, and thus improve the effect of vulnerability detection.

5.5.3 RQ3: Is the explicit inclusion of threat code slices in GPTVD’s COTs effective?

The explicit inclusion of threat code slices in GPTVD’s Code-of-Threats (COTs) is intended to strengthen vulnerability detection by explicitly specifying relevant code segments that represent potential threats. This approach recognizes that certain code patterns or sequences are more likely to contain vulnerabilities and focuses on them during the detection process.

To evaluate the effectiveness of the explicit inclusion of threat code slices in GPTVD’s COTs, we conducted experiments, comparing GPTVD’s performance with and without the explicit inclusion of threat code slices. The results are shown in TABLE 5.

Table 5 GPTVD’s performance with and without the explicit inclusion of threat code slices.

Method	Accuracy (%)	Precision (%)	Recall (%)	F1
w/o slices	84.28	80.30	91.26	0.854
GPTVD	90.72	86.71	96.39	0.913

From the experimental results, it can be seen that the explicit prompt of data flow control flow can improve the performance of GPTVD, as shown in Fig. 6, line 67 is the threat code. This function copies “data” to “dest”, so it is very important to determine the size of “data” and “dest”. When the explicit prompt of data flow and control flow is removed, ChatGPT does not obtain the size of “data” according to 51 lines, which leads to missed detection, but GPTVD successfully detected potential vulnerabilities because it sensed the size of the “data” and the size of the “data” was greater than the size of “dest”.

```
# CWE121_Stack_Based_Buffer_Overflow__CWE806_char_declare_ncat_10.c
## Source Code:
48 static void goodG2B1()
49 {
50     char * data;
51     char dataBuffer[100];
52     data = dataBuffer;
53     if(globalFalse)
54     {
55         /* INCIDENTAL: CWE 561 Dead Code, the code below will never run */
56         printf("Benign, fixed string");
57     }
58     else
59     {
60         /* FIX: Initialize data as a small buffer that as small or smaller than the small buffer used in the sink */
61         memset(data, 'A', 50-1); /* fill with 'A's */
62         data[50-1] = '\0'; /* null terminate */
63     }
64     {
65         char dest[50] = "";
66         /* POTENTIAL FLAW: Possible buffer overflow if data is larger than sizeof(dest)-strlen(dest)*/
67         strcat(dest, data, strlen(data));
68         dest[50-1] = '\0'; /* Ensure the destination buffer is null terminated */
69     }
70     [strcat] at line [67]
71 }
72
73 ## Question: Does the Threat Code cause a vulnerability?
74 ## w/o slices answer: Here's the analysis of the code: (1) At line [52], 'data' is set to 'dataBuffer'. (2) At line [53], a conditional statement checks if 'globalFalse' is false. If it is, the code proceeds to line [60]. (3) At line [60], the function 'memset' is used to fill 'data' with '50-1' copies of the char 'A'. (4) At line [61], the last element of 'data' is set to the null terminator '\0', indicating the end of 'data'. (5) At line [67], the function 'strcat' is used to concatenate 'strlen(data)' chars of 'data' to 'dest'. 'dest' has the size of '50' chars. 'data' has the size of '50' chars. 'dest' is enough to hold 'data'. Therefore, no buffer overflow vulnerability is present. (6) At line [68], the element of 'dest' with the index of '50-1' is set to '\0'. [Analysis] Therefore, in this code snippet, the Threat Code does not cause a potential vulnerability.
75
76 ## COTVD Answer: Here's the analysis of the code: At line [52], 'data' is set to 'dataBuffer' with a size of '100' chars. A conditional statement checks if 'globalFalse' is true. If it is, the code proceeds to line [61]. At line [61], the function 'memset' is used to fill 'data' with '50-1' copies of the char 'A'. At line [62], the last element of 'data' is set to the null terminator '\0', indicating the end of 'data'. At line [67], the function 'strcat' is used to concatenate 'strlen(data)' chars of 'data' to 'dest'. 'dest' has the size of '50' chars. 'data' has the size of '100' chars. 'dest' is not enough to hold 'data'. So there is a buffer overflow vulnerability. At line [68], the element of 'dest' with the index of '50-1' is set to '\0'. Therefore, in this code snippet, the Threat Code causes a potential vulnerability.
```

Fig. 6 Explicit threat code slice prompts ablation experimental results.

5.5.4 RQ4: Does the human expert analysis of data flow and control flow processes in GPTVD contribute to vulnerability detection?

The human expert analysis of data flow and control flow processes in GPTVD plays a crucial role in facilitating vulnerability detection. By involving human experts in the analysis, GPTVD benefits from their expertise and knowledge in identifying potential vulnerabilities related to data and control dependencies. To validate the effectiveness of human expert analysis, we designed comparative experiments between GPTVD with human expert analysis and GPTVD without human expert analysis. The experimental results are presented in TABLE 6.

Table 6 The result of RQ4.

Method	Accuracy (%)	Precision (%)	Recall (%)	F1
w/o analysis by human experts	75.94	71.84	86.13	0.783
GPTVD	90.72	86.71	96.39	0.913

From the results, it can be seen that human expert analysis can significantly improve the performance of GPTVD vulnerability detection. Among them, accuracy and precision have increased by nearly 15 percentage points, and recall has increased by 10 percentage points. It shows that the analysis of threat code data dependence and control dependence by human experts can help LLM deeply understand the causes of vulnerabilities, and is obviously helpful to the reasoning of vulnerability detection.

6 Failure analysis

We summarized the samples of GPTVD failures and divided them into three categories. The first type is the failure caused by the lack of key variable information. The second type is the failure caused by the error of model mathematical and logical reasoning. Finally, when the model does not generate the answer in the expected form, it may also lead to detection failure.

Variable size unknown failure. As shown in Fig. 7, “data” is passed in as a parameter of the function, and its size is unknown. Normally, the program needs to check the “data” to ensure that no vulnerabilities will occur. GPTVD directly concluded that there were no vulnerabilities in the sample, resulting in missed inspection.

```

Variable size unknown error.
# CWE122_Heap_Based_Buffer_Overflow__c_dest_char_cat_51b.c
## Lines of (Data Flow and Control Flow) about Threat Code
25     void CWE122_Heap_Based_Buffer_Overflow__c_dest_char_cat_51b_badSink(char * data)
26     {
27         char source[100];
28         memset(source, 'C', 100-1);
29         source[100-1] = '\0';
30         strcat(data, source);
31         printf("data: %s\n", data);
32         free(data);
33     }
34
## Question: Is there a vulnerability in (Data Flow and Control Flow)?
## Answer: Here's the analysis of the code: (1) At line [28], the function 'memset' is used to fill 'source' with '100-1' copies of the char 'C'. (2) At line [29], the last element of 'source' is set to the null terminator '\0', indicating the end of 'source'. (3) At line [32], the function 'strcat' is used to concatenate 'source' to 'data'. The potential vulnerability is checked at runtime by comparing the size of 'source' with the available space in 'data' (i.e., 'sizeof(data)-strlen(data)'). Therefore, the threat code [memset] at line [29] does not cause a vulnerability.

Mathematical and logical reasoning error.
# CWE122_Heap_Based_Buffer_Overflow__c_CWE193_char_memcpy_14.c
## Lines of (Data Flow and Control Flow) about Threat Code
28     void CWE122_Heap_Based_Buffer_Overflow__c_CWE193_char_memcpy_14_bad()
29     {
30         char * data;
31         data = NULL;
32         if(globalFive==5)
33         {
34             data = (char *)malloc(10*sizeof(char));
35             char source[10+1] = SRC_STRING;
36             memcpy(data, source, (strlen(source) + 1) * sizeof(char));
37             free(data);
38         }
39     }
40
## Question: Is there a vulnerability in (Data Flow and Control Flow)?
## Answer: Here's the analysis of the code: (1) At line [31], the pointer 'data' is initialized to NULL. (2) At line [32], a conditional statement checks if 'globalFive' is equal to 5. If it is, the code proceeds to line [35]. (3) At line [35], the function 'malloc' is used to allocate 'data' with a size of '10' chars. (4) At line [38], 'source' with a size of '10+1' chars is created and initialized with the value defined by 'SRC_STRING'. (5) At line [41], the function 'memcpy' is used to copy 'strlen(source) + 1' * sizeof(char) bytes from 'source' to 'data'. 'data' has a size of '10' chars, and 'source' has a size of '10+1' chars, so 'data' is enough to hold 'source'. (6) At line [42], 'data' is printed. (7) At line [43], the function 'free' is used to free 'data'. [Analysis] In this code snippet, there is no potential vulnerability. Therefore, the threat code [free] at line [43] does not cause a vulnerability.

```

Fig. 7 Fail analysis.

Mathematical and logical reasoning failure. As shown in Fig. 7, GPTVD makes it clear that the size of the “data” is 10 and the size of the “source” is $10 + 1 = 11$.

However, due to mathematical and logical reasoning errors, the wrong belief that “data” is enough to accommodate the “source”, resulting in missed detection.

Generation form failure. When ChatGPT does not generate answers in the form expected by prompt, it may also lead to the failure of GPTVD.

7 Threats to Validity

We will discuss threats to validity from three perspectives: internal, external, and construct validity threats.

7.1 Internal validity threats

Dataset bias: Due to the limitation of LLM reasoning time and economic cost, the experiments were conducted on a single dataset, which may have biases that affect the generalizability. We will expand the scale of the experimental dataset in the future.

Code language bias: Currently, GPTVD only supports C/C++ code. The effectiveness of support for other languages such as Java and Python needs further evaluation. However, due to the similarity of programming languages, this threat is limited. There is also a bias towards specific vulnerability types, with a focus on buffer overflow vulnerabilities. The support for other types such as SQL injection, XSS, etc., needs further verification. Considering that GPTVD employs data flow and control flow analysis to infer vulnerabilities, it has a certain level of generality, thus mitigating this threat to some extent.

7.2 External validity threats

The subjective nature of manually constructing prompts introduces subjectivity, and different experts may have variances in their code flow analysis results. However, experiments have shown that the analysis process conducted by human experts significantly improves the performance of GPTVD. Therefore, this threat is limited in its impact.

7.3 Construct validity threats

Scalability: The feasibility of applying GPTVD to large code repositories needs further evaluation of the model’s scalability. In general, although the method has shown good performance, conducting more systematic experiments in broader and more complex real-world scenarios is necessary to further validate its effectiveness and limitations.

8 Conclusion

GPTVD is the first attempt within the LLM in-context learning paradigm to perform vulnerability reasoning and detection. By leveraging the contextual learning and reasoning capabilities of large-scale language models, GPTVD achieves vulnerability detection at the slice level and provides a detailed analysis of the causes of vulnerabilities. Experiments on public datasets demonstrate that GPTVD outperforms existing state-of-the-art methods in terms of accuracy, recall, and F1 score for vulnerability detection. In summary, GPTVD offers an effective and interpretable approach for

slice-level vulnerability detection. Future work will focus on addressing its limitations and further validating the method in broader and more complex real-world scenarios, including expanding the dataset and exploring the applicability to other types of vulnerabilities and developing more efficient COT construction methods.

References

- [1] Steenhoek, B., Rahman, M.M., Jiles, R., Le, W.: An empirical study of deep learning models for vulnerability detection. In: Proceedings of the 45th International Conference on Software Engineering. ICSE '23, pp. 2237–2248. IEEE Press, ??? (2023). <https://doi.org/10.1109/ICSE48619.2023.00188> . <https://doi.org/10.1109/ICSE48619.2023.00188>
- [2] Gao, J., Jiang, Y., Liu, Z., Yang, X., Wang, C., Jiao, X., Yang, Z., Sun, J.: Semantic learning and emulation based cross-platform binary vulnerability seeker. *IEEE Transactions on Software Engineering* **47**(11), 2575–2589 (2021) <https://doi.org/10.1109/TSE.2019.2956932>
- [3] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. NIPS'17, pp. 6000–6010. Curran Associates Inc., Red Hook, NY, USA (2017)
- [4] Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y.: Vuldeepecker: A deep learning-based system for vulnerability detection (2018)
- [5] Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. Curran Associates Inc., Red Hook, NY, USA (2019)
- [6] Li, Y., Wang, S., Nguyen, T.N.: Vulnerability detection with fine-grained interpretations. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2021, pp. 292–303. Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3468264.3468597> . <https://doi.org/10.1145/3468264.3468597>
- [7] Hin, D., Kan, A., Chen, H., Babar, M.A.: Linevd: Statement-level vulnerability detection using graph neural networks. In: 2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR), pp. 596–607 (2022). <https://doi.org/10.1145/3524842.3527949>
- [8] Dong, Q., Li, L., Dai, D., Zheng, C., Wu, Z., Chang, B., Sun, X., Xu, J., Li, L., Sui, Z.: A Survey on In-context Learning (2023)
- [9] Zou, D., Wang, S., Xu, S., Li, Z., Jin, H.: μ vuldeepecker: A deep learning-based system for multiclass vulnerability detection. *IEEE Transactions on Dependable*

- and Secure Computing **18**(5), 2224–2236 (2021) <https://doi.org/10.1109/TDSC.2019.2942930>
- [10] Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Computation* **9**(8), 1735–1780 (1997) <https://doi.org/10.1162/neco.1997.9.8.1735>
 - [11] Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z.: Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* **19**(4), 2244–2258 (2022) <https://doi.org/10.1109/TDSC.2021.3051525>
 - [12] Cheng, X., Wang, H., Hua, J., Xu, G., Sui, Y.: Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Trans. Softw. Eng. Methodol.* **30**(3) (2021) <https://doi.org/10.1145/3436877>
 - [13] Wang, H., Ye, G., Tang, Z., Tan, S.H., Huang, S., Fang, D., Feng, Y., Bian, L., Wang, Z.: Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Transactions on Information Forensics and Security* **16**, 1943–1958 (2021) <https://doi.org/10.1109/TIFS.2020.3044773>
 - [14] Shiralinasab Langari, A., Yeganeh, L., Khoa Nguyen, K.: Grothendieck graph neural network (ggnn): A path-based framework for network modelling. In: *GLOBE-COM 2023 - 2023 IEEE Global Communications Conference*, pp. 6548–6553 (2023). <https://doi.org/10.1109/GLOBECOM54140.2023.10437032>
 - [15] Li, Z., Zou, D., Xu, S., Chen, Z., Zhu, Y., Jin, H.: Vuldeelocator: A deep learning-based fine-grained vulnerability detector. *IEEE Transactions on Dependable and Secure Computing* **19**(4), 2821–2837 (2022) <https://doi.org/10.1109/TDSC.2021.3076142>
 - [16] Pornprasit, C., Tantithamthavorn, C.K.: Deeplinedp: Towards a deep learning approach for line-level defect prediction. *IEEE Transactions on Software Engineering* **49**(1), 84–98 (2023) <https://doi.org/10.1109/TSE.2022.3144348>
 - [17] Zou, D., Zhu, Y., Xu, S., Li, Z., Jin, H., Ye, H.: Interpreting deep learning-based vulnerability detector predictions based on heuristic searching. *ACM Trans. Softw. Eng. Methodol.* **30**(2) (2021) <https://doi.org/10.1145/3429444>
 - [18] Ying, R., Bourgeois, D., You, J., Zitnik, M., Leskovec, J.: GNNExplainer: generating explanations for graph neural networks. Curran Associates Inc., Red Hook, NY, USA (2019)
 - [19] Wang, C., Yang, Y., Gao, C., Peng, Y., Zhang, H., Lyu, M.R.: No more

- fine-tuning? an experimental evaluation of prompt tuning in code intelligence. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2022, pp. 382–394. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3540250.3549113> . <https://doi.org/10.1145/3540250.3549113>
- [20] Wang, C., Yang, Y., Gao, C., Peng, Y., Zhang, H., Lyu, M.R.: Prompt tuning in code intelligence: An experimental evaluation. *IEEE Trans. Softw. Eng.* **49**(11), 4869–4885 (2023) <https://doi.org/10.1109/TSE.2023.3313881>
- [21] Peng, Y., Wang, C., Wang, W., Gao, C., Lyu, M.R.: Generative type inference for python. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 988–999 (2023). <https://doi.org/10.1109/ASE56229.2023.00031>
- [22] Xu, J., Cui, Z., Zhao, Y., Zhang, X., He, S., He, P., Li, L., Kang, Y., Lin, Q., Dang, Y., Rajmohan, S., Zhang, D.: Unilog: Automatic logging via llm and in-context learning. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE ’24. Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3597503.3623326> . <https://doi.org/10.1145/3597503.3623326>